

Technical Interview Assignment: Arti Bhargava

Interview Assignment:

Build a Secured Multi-Tenant Resource Booking Platform & Automated Security Audit Pipeline

Objective

Build a secure, scalable backend API with Django or FastAPI and an interactive React JS frontend that allows users to manage resort/resource bookings, handle cloud file storage, enforce role-based access control, and log automated security audit trails.

Total Time: 3 Hours

1. Application Scenario

Build a dual-purpose operational backend and UI that manages property/resort reservations and resource allocations (e.g., thebharatresort.com) while integrating threat checking and automated security log auditing for user transactions.

2. Core Backend & REST APIs

Develop using **Django REST Framework (DRF)** or **FastAPI**:

- POST /api/auth/token (Issue secure JWT access & refresh tokens with RBAC scope validation)
- POST /api/bookings (Create resource/resort bookings with date conflict checks and rate calculation)
- GET /api/bookings/my-reservations (Retrieve filtered reservations for authenticated users)
- POST /api/security/audit-log (Record security events, query logs, and flag suspicious request payloads)

3. Database & Data Architecture

Use **PostgreSQL** or **MySQL** with Django ORM / SQLAlchemy:

- User: id, full_name, email, password_hash, role (ADMIN/STAFF/CUSTOMER), created_at
- Resource: id, name, category (ROOM/SUITE/HALL), daily_rate, is_available
- Booking: id, resource_id, user_id, check_in, check_out, total_price, status (CONFIRMED/CANCELLED)

- **AuditLog:** id, user_id, endpoint, ip_address, threat_level (LOW/MEDIUM/HIGH), timestamp

4. Security & Business Workflows

- **JWT & OAuth2 Authentication Engine:** Implement end-to-end token validation, password hashing, and role-based route permissions to protect administrative endpoints.
- **Automated Threat Logging:** Build middleware or a service worker that analyzes incoming request payloads for potential security risks, tagging high-risk activity in the audit logs.

5. Frontend & UI Implementation

- **React JS Booking & Audit Dashboard:** Build an interactive resort booking interface and a real-time security log viewer.
- **Dynamic Filters & Forms:** Enable users to filter available resources by date/category and allow admins to search security audit trails by threat level.

6. Time Allocation & Evaluation

- **Recommended Time Allocation:**
 - Backend APIs & JWT Auth Architecture (DRF/FastAPI): 40 min
 - Reservation Business Logic & Threat Audit Engine: 40 min
 - React JS Frontend Dashboard & Interactive Forms: 45 min
 - Database Query Optimization & Pytest Unit Tests: 20 min
 - Docker & Cloud Storage/Deployment Setup: 20 min
 - Buffer: 15 min
- **Evaluation Criteria:**
 - Clean RESTful API Design & DRF/FastAPI Code Quality (25%)
 - Authentication, Authorization & Security Best Practices (25%)
 - Database Design & Transaction Safety (20%)
 - React JS Component Structure & State Handling (15%)
 - Unit Testing Coverage & Docker Deployment Readiness (15%)